IPC - Linux

0. SYSTEM V IPC vs POSIX IPC
1. PIPES and NAMED PIPES
2. SHARED MEMORY and MEMORY MAPPING
3. MESSAGE QUEUES
4. SEMAPHORES and NAMED SEMAPHORES
5. SIGNALS
6. SOCKET

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux

0. SYSTEM V IPC vs POSIX IPC
1. PIPES and NAMED PIPES
2. SHARED MEMORY and MEMORY MAPPING
3. MESSAGE QUEUES
4. SEMAPHORES and NAMED SEMAPHORES
5. SIGNALS
6. MEMORY MAPPING
7. SOCKET

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux

System V IPC vs POSIX IPC

System V	POSIX (Portable Operating System Interface)
AT & T	**IEEE**
Message Queue msgrcv, msgsnd, msgget	**Message Queue** mq_close, mq_getsetattr, mq_open, mq_receive mq_setattr, mq_timedsend, mq_getattr, mq_notify, mq_overview, mq_send, mq_timedreceive, mq_unlink. `linux manual: man mq_overview`
Semaphore semop, semctl, semget	**Semaphore** sem_close, sem_getvalue, sem_open, sem_post, sem_trywait, sem_wait, sem_destroy, sem_init sem_overview, sem_timedwait, sem_unlink. `linux manual: man sem_overview`
Shared Memory shmat, shmdt, shmget	**Shared Memory** shm_open, ftruncate, mmap, munmap, shm_unlink, fstat, close, fchown, fchmod `linux manual: man shm_overview`

IPC - Linux
IPCS-System V

```
IPCS(1)                                User Commands                                IPCS(1)

NAME
       ipcs - show information on IPC facilities

SYNOPSIS
       ipcs [options]

DESCRIPTION
       ipcs  shows information on System V inter-process communication facilities.  By default
       it shows information about all three resources: shared memory segments, message queues,
       and semaphore arrays.

OPTIONS
       -i, --id id
              Show  full  details on just the one resource element identified by id.  This op-
              tion needs to be combined with one of the three resource options: -m, -q or -s.

       -h, --help
              Display help text and exit.

       -V, --version
              Display version information and exit.

   Resource options
       -m, --shmems
              Write information about active shared memory segments.

       -q, --queues
              Write information about active message queues.

       -s, --semaphores
              Write information about active semaphore sets.

       -a, --all
              Write information about all three resources (default).
```

IPC - Linux
IPCS-System V

```
------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x51011ceb 15         asp        600        3208       1
0x51011d38 19         asp        600        8          1

------ Semaphore Arrays --------
key        semid      owner      perms      nsems
0x51010879 0          asp        600        1
0x51011ce9 2          asp        600        1
0x51011d03 3          asp        600        1
```

IPC - Linux

0. IPCS
1. PIPES and NAMED PIPES
2. SHARED MEMORY and MEMORY MAPPING
3. MESSAGE QUEUES
4. SEMAPHORES and NAMED SEMAPHORES
5. SIGNALS
6. SOCKET

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux

Pipes :

- Used for communication between related processes
- Are unidirectional

Named pipes :

- Can be used for communication between unrelated processes
- Are bidirectional

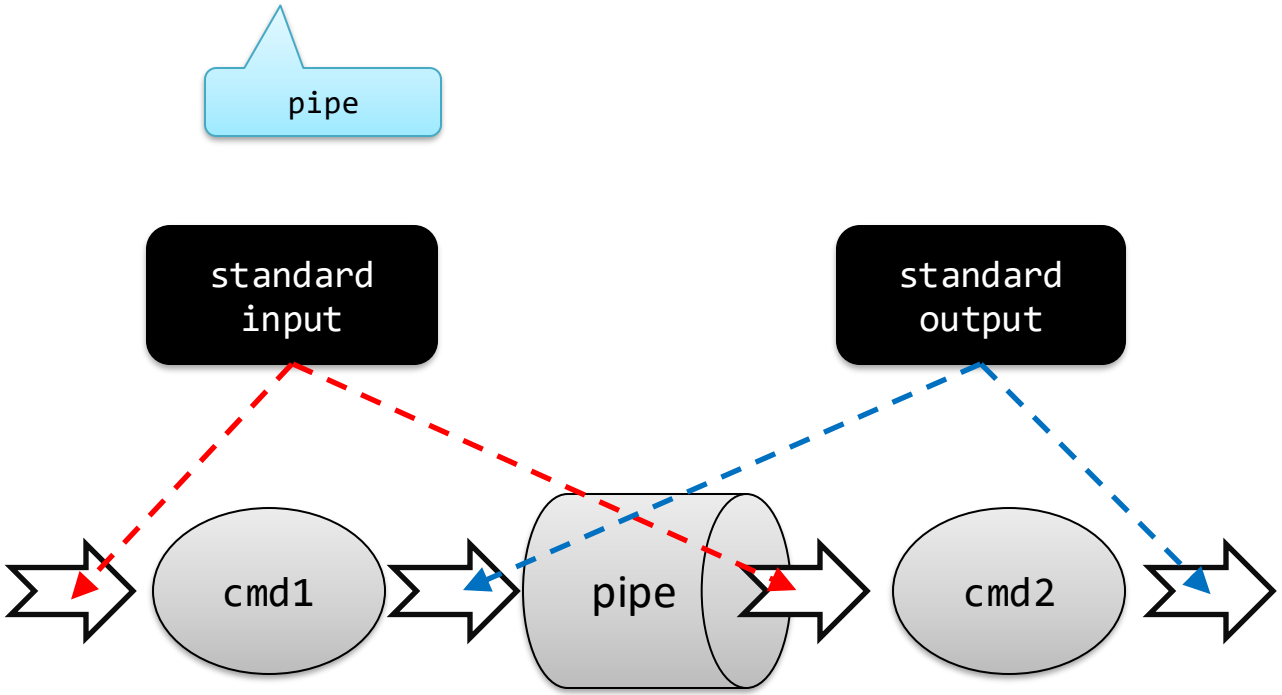
Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux
PIPES

A pipe is a unidirectional data channel that is used for interprocess communication.

e.g.

```
echo hello world  | wc -c
```



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC – Linux (PIPES)

PIPE(2)

NAME

pipe, pipe2 - create pipe

```c
#include<stdio.h>
#include<unistd.h>

int main() {
    int pipe_fds[2];
    int return_status;
    return_status = pipe(pipe_fds);
    printf("reading file descriptor : %d\n", pipe_fds[0]);
    printf("writing file descriptor : %d\n", pipe_fds[1]);
    printf("return status           : %d\n", return_status);

    if (return_status == -1) {
        printf(" Pipe creation is failed.\n");
        return 1;
    }
    return 0;
}

```

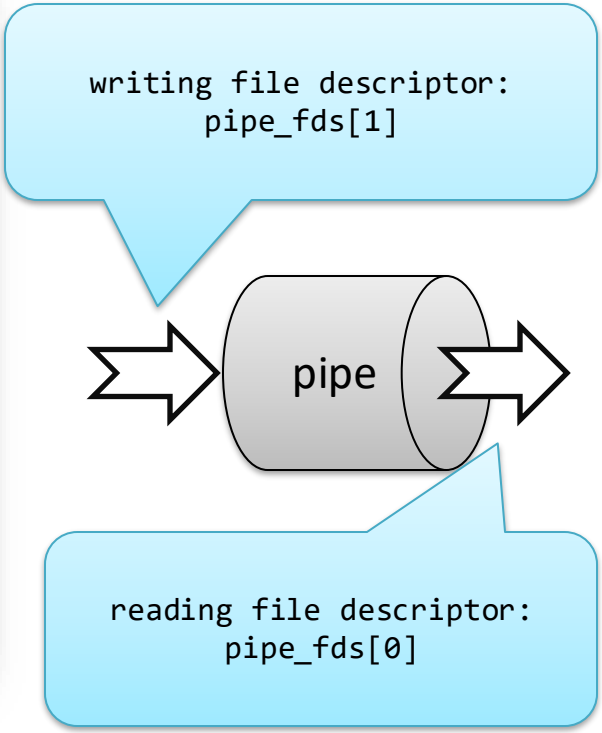
IPC – Linux (PIPES)

```c
#include<stdio.h>
#include<unistd.h>

int main() {
    int pipe_fds[2];
    int return_status;
    return_status = pipe(pipe_fds);
    printf("reading file descriptor : %d\n", pipe_fds[0]);
    printf("writing file descriptor : %d\n", pipe_fds[1]);
    printf("return status           : %d\n", return_status);

    if (return_status == -1) {
        printf(" Pipe creation is failed.\n");
        return 1;
    }
    return 0;
}
```

two file descriptors: read and write

pipe system call

```
reading file descriptor : 3
writing file descriptor : 4
return status           : 0
```

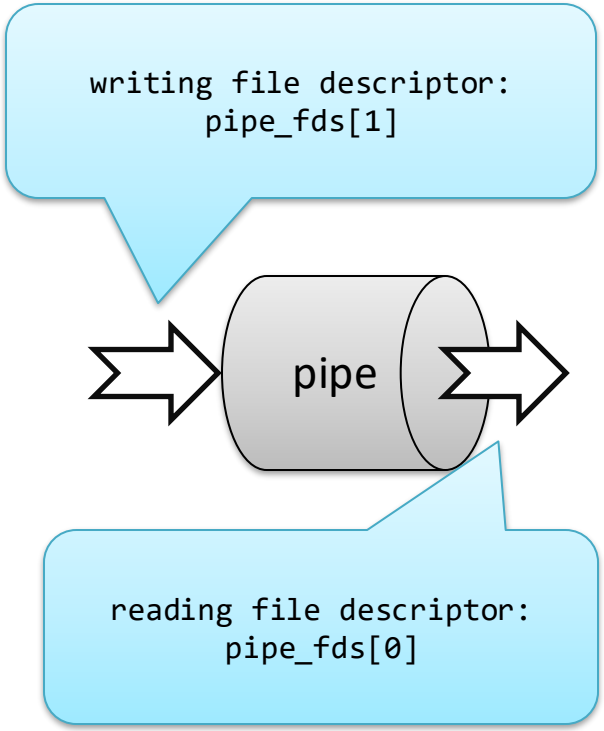


IPC – Linux (PIPES)

```c
#include<stdio.h>
#include<unistd.h>

int main() {
    int pipe_fds[2];
    int return_status;
    char write_messages[2][30] = {
        "Hello from user 1",
        "Hello from user 2"
    };
    char read_message[30];
    return_status = pipe(pipe_fds);
    if (return_status == -1) {
        printf(" Pipe creation is failed.\n");
        return 1;

    pr[illegible] Writing to pipe   : Msg 1 is %s\n", write_messages[0]);
    // [illegible]iting to pipe Msg 1
    write(pipe_fds[1], write_messages[0], sizeof(write_messages[0]));
    // Reading from pipe Msg 1
    read(pipe_fds[0], read_message, sizeof(read_message));
    pr[illegible]f("Reading from pipe : Msg 1 is %s\n", read_message);
    [illegible]

    [illegible] Msg 2 is %s\n", write_messages[1]);
    // Writing to pipe, Msg 2
    write(pipe_fds[1], write_messages[1], sizeof(write_messages[1]));
    // Reading from pipe, Msg 2
    read(pipe_fds[0], read_message, sizeof(read_message));
    printf("Reading from pipe : Msg 2 is %s\n", read_message);

    return 0;
}
```

write system call

read system call



IPC – Linux (PIPES)

```c
#include<stdio.h>
#include<unistd.h>

int main() {
    int pipe_fds[2];
    int return_status;
    char write_messages[2][30] = {
        "Hello from user 1",
        "Hello from user 2"
    };
    char read_message[30];
    return_status = pipe(pipe_fds);
    if (return_status == -1) {
        printf(" Pipe creation is failed.\n");
        return 1;
    }

    printf("Writing to pipe   : Msg 1 is %s\n", write_messages[0]);
    // Writing to pipe Msg 1
    write(pipe_fds[1], write_messages[0], sizeof(write_messages[0]));
    // Reading from pipe Msg 1
    read(pipe_fds[0], read_message, sizeof(read_message));
    printf("Reading from pipe : Msg 1 is %s\n", read_message);
    printf("\n");

    printf("Writing to pipe   : Msg 2 is %s\n", write_messages[1]);
    // Writing to pipe, Msg 2
    write(pipe_fds[1], write_messages[1], sizeof(write_messages[1]));
    // Reading from pipe, Msg 2
    read(pipe_fds[0], read_message, sizeof(read_message));
    printf("Reading from pipe : Msg 2 is %s\n", read_message);

    return 0;
}
```

```
Writing to pipe   : Msg 1 is Hello from user 1
Reading from pipe : Msg 1 is Hello from user 1

Writing to pipe   : Msg 2 is Hello from user 2
Reading from pipe : Msg 2 is Hello from user 2
```

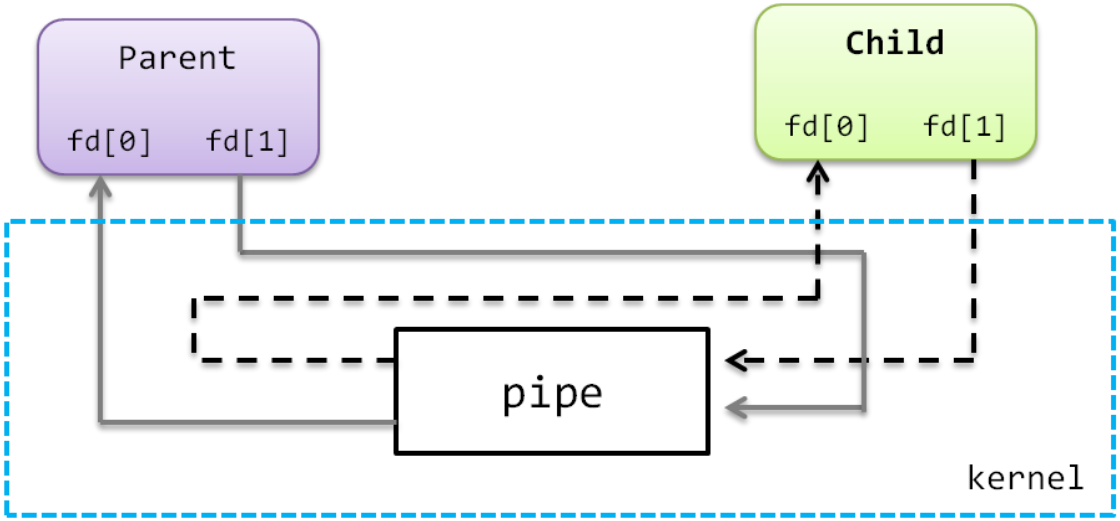
IPC – Linux (PIPES)

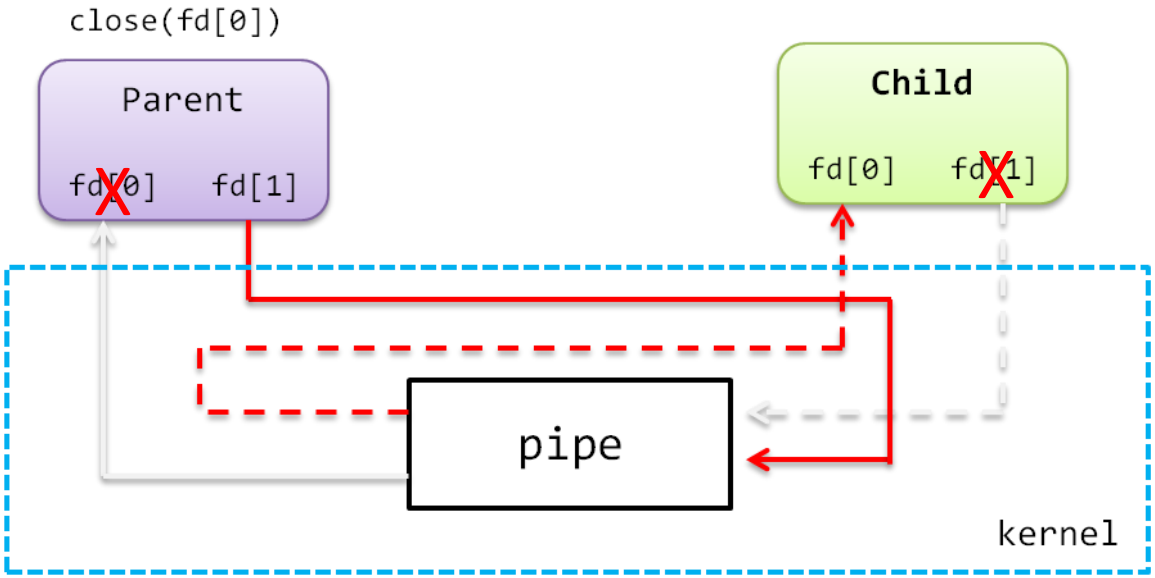
```c
#include<stdio.h>
#include<unistd.h>

int main() {
    int pipe_fds[2];
    int return_status;
    char write_messages[2][30] = {
        "Hello from user 1",
        "Hello from user 2"
    };
    char read_message[30];
    return_status = pipe(pipe_fds);
    if (return_status == -1) {
        printf(" Pipe creation is failed.\n");
        return 1;
    }

    printf("Writing to pipe   : Msg 1 is %s\n", write_messages[0]);
    // Writing to pipe Msg 1
    write(pipe_fds[1], write_messages[0], sizeof(write_messages[0]));
    // Reading from pipe Msg 1
    read(pipe_fds[0], read_message, sizeof(read_message));
    printf("Reading from pipe : Msg 1 is %s\n", read_message);
    printf("\n");

    printf("Writing to pipe   : Msg 2 is %s\n", write_messages[1]);
    // Writing to pipe, Msg 2
    write(pipe_fds[1], write_messages[1], sizeof(write_messages[1]));
    // Reading from pipe, Msg 2
    read(pipe_fds[0], read_message, sizeof(read_message));
    printf("Reading from pipe : Msg 2 is %s\n", read_message);

    return 0;
}
```

MSG 1

MSG 2

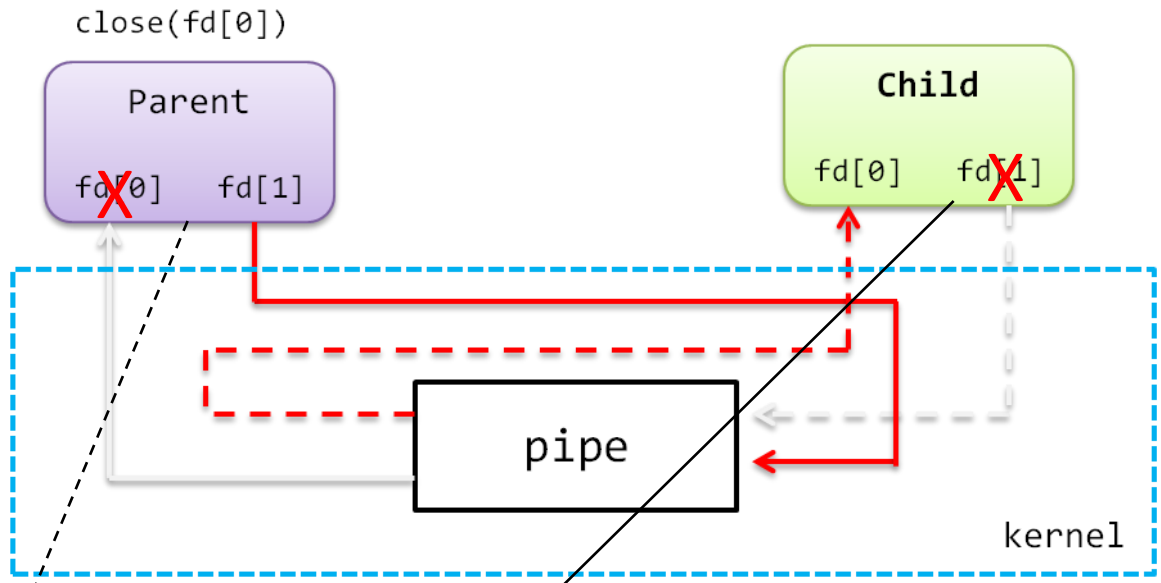
IPC – Linux (PIPES)





IPC – Linux (PIPES)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <wait.h>

#define BUFFER_SIZE 256

int main() {
    int pipefd[2];
    char buffer[BUFFER_SIZE];
    pid_t pid;

    if (pipe(pipefd) == -1) {
        perror("Pipe creation failed");
        exit(EXIT_FAILURE);
    }
    pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        exit(EXIT_FAILURE);
    }
    if (pid == 0) {
        close(pipefd[1]);
        int num_bytes = read(pipefd[0], buffer, BUFFER_SIZE);
        if (num_bytes > 0) {
            printf("Child: Received message from parent: %s\n", buffer);
        }
        close(pipefd[0]);
        exit(EXIT_SUCCESS);
    } else {
        close(pipefd[0]);
        const char* message = "Hello from parent!";
        size_t num_bytes = write(pipefd[1], message, strlen(message) + 1);
        close(pipefd[1]);
        wait(NULL);
        exit(EXIT_SUCCESS);
    }
    return 0;
}
```




```
Child: Received message from parent: Hello from parent!
```

IPC - Linux
NAMED PIPES (FIFO)

Named pipes are used for interprocess communication. They are device files.

mkfifo

```
MKFIFO(1)                                                User Commands

NAME
       mkfifo - make FIFOs (named pipes)

SYNOPSIS
       mkfifo [OPTION]... NAME...

DESCRIPTION
       Create named pipes (FIFOs) with the given NAMEs.
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux
NAMED PIPES (FIFO)

mknod

```
MKNOD(1)                                                User Commands

NAME
       mknod - make block or character special files

SYNOPSIS
       mknod [OPTION]... NAME TYPE [MAJOR MINOR]

DESCRIPTION
       Create the special file NAME of the given TYPE.
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux
NAMED PIPES (FIFO)

mkfifo

name

```
~$ mkfifo FIFO
~$ ls -al | grep FIFO
prw-r--r-- 1 user user    0 Jun 12 03:41 FIFO
~$
```

named pipe

```
~$
~$ pstree -p -T > FIFO
~$
~$
~$
```

❶

```
~$
~$ cat < FIFO
tini(1)---sh(7)---node(8)-+-bash(575)---pstree(960)
                          |-bash(928)---cat(972)
                          `-sshd(23)
```

❷

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux
NAMED PIPES (FIFO)

mknod

```
MKNOD(2)                                Linux Programmer's Manual

NAME
       mknod, mknodat - create a special or ordinary file

SYNOPSIS
       #include <sys/types.h>
       #include <sys/stat.h>
       #include <fcntl.h>
       #include <unistd.h>

       int mknod(const char *pathname, mode_t mode, dev_t dev);
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux
NAMED PIPES (FIFO)

mknod

name

type (named pipe)

```
~$
~$ mknod FIFO_NOD  p
~$ ls -al | grep FIFO_NOD
prw-r--r-- 1 user user    0 Jun 12 03:52 FIFO_NOD
~$
```

named pipe

```
~$
~$ ls -al > FIFO_NOD
~$
~$
```

❶

```
~$
~$ wc -w < FIFO_NOD
123
~$
```

❷

IPC - Linux
NAMED PIPES (FIFO)

Mknod

```
MKNOD(2)                    Linux Programmer's Manual                    MKNOD(2)

NAME
       mknod, mknodat - create a special or ordinary file

SYNOPSIS
       #include <sys/types.h>
       #include <sys/stat.h>
       #include <fcntl.h>
       #include <unistd.h>

       int mknod(const char *pathname, mode_t mode, dev_t dev);

       #include <fcntl.h>           /* Definition of AT_* constants */
       #include <sys/stat.h>

       int mknodat(int dirfd, const char *pathname, mode_t mode, dev_t dev);
```

IPC - Linux
NAMED PIPES (FIFO)

Mkfifo

```
MKFIFO(3)                    Linux Programmer's Manual                    MKFIFO(3)

NAME
       mkfifo, mkfifoat - make a FIFO special file (a named pipe)

SYNOPSIS
       #include <sys/types.h>
       #include <sys/stat.h>

       int mkfifo(const char *pathname, mode_t mode);

       #include <fcntl.h>           /* Definition of AT_* constants */
       #include <sys/stat.h>

       int mkfifoat(int dirfd, const char *pathname, mode_t mode);
```

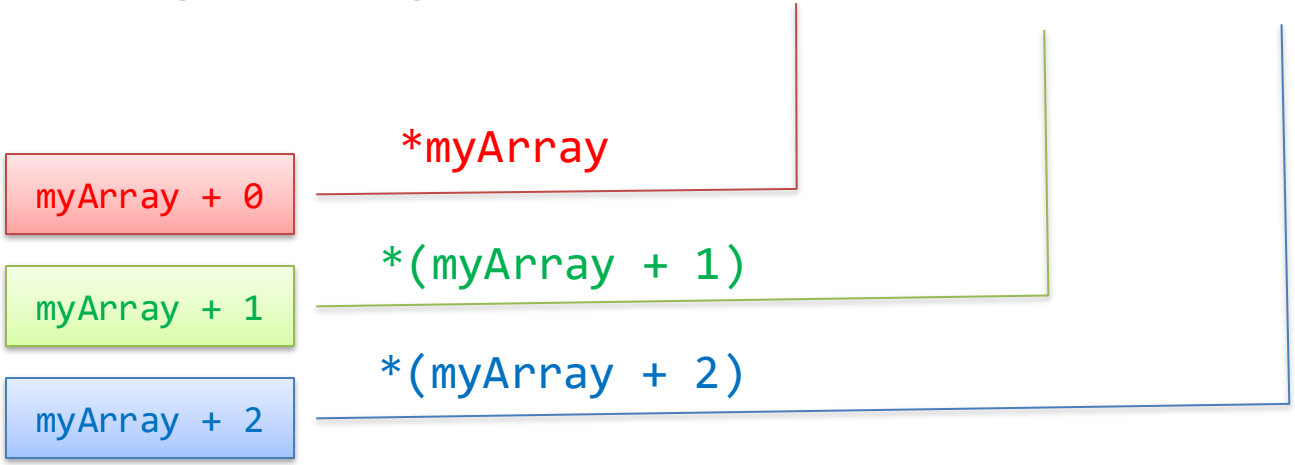
IPC - Linux

0. IPCS
1. PIPES and NAMED PIPES
2. SHARED MEMORY and MEMORY MAPPING
3. MESSAGE QUEUES
4. SEMAPHORES and NAMED SEMAPHORES
5. SIGNALS
6. MEMORY MAPPING
7. SOCKET

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Arrays in C

```
int     myArray[3]={100, 200, 300}
```



Arrays in C

```c
#include <stdio.h>
int main()
{
    int myArray[3] = {100, 200, 300};

    printf("\n");
    printf("\t %d] %p %d \n", __LINE__, myArray, *myArray);
    printf("\t %d] %p %d \n", __LINE__, myArray+1, *(myArray+1));
    printf("\t %d] %p %d \n", __LINE__, myArray+2, *(myArray+2));
    printf("\n");
    printf("\t %d] %p %d \n", __LINE__, &myArray[0], myArray[0]);
    printf("\t %d] %p %d \n", __LINE__, &myArray[1], myArray[1]);
    printf("\t %d] %p %d \n", __LINE__, &myArray[2], myArray[2]);

    return 0;
}
```

```
 7] 0x7fff822ae07c 100
 8] 0x7fff822ae080 200
 9] 0x7fff822ae084 300

 11] 0x7fff822ae07c 100
 12] 0x7fff822ae080 200
 13] 0x7fff822ae084 300
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux
Memory Mapping

mmap() creates a new mapping in the virtual address space of the calling process.

- **File mapping:**

  reading or writing to those areas of memory causes the file to be read or written.

- **Anonymous mapping:**

  it is like dynamic memory allocation(malloc) and the contents are initialized to zero.

IPC - Linux
Memory Mapping

```
MMAP(2)                    Linux Programmer's Manual

NAME
       mmap, munmap - map or unmap files or devices into memory

SYNOPSIS
       #include <sys/mman.h>

       void *mmap(void *addr, size_t length, int prot, int flags,
                  int fd, off_t offset);
       int munmap(void *addr, size_t length);
```

IPC - Linux
Memory Mapping

return

the starting address of the mapping on success or MAP_FAILED on error

addr(virtual address)

user specified or generated by the kernel (addr = NULL)

length

the size of mapping in bytes

prot

it is either PROT_NONE or the bitwise OR of one or more of the following flags:

- PROT_EXEC Pages may be executed
- PROT_READ Pages may be read
- PROT_WRITE Pages may be written
- PROT_NONE Pages may not be accessed

flags

mapping type (MAP_SHARED or MAP_PRIVATE)

fd

the file descriptor identifying the file to be mapped

offset

the starting point of the file, e.g. offset is 0 if the entire file is mapped

IPC - Linux
Memory Mapping

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <ctype.h>

int main() {
    int fd;
    struct stat file_info;
    char *mapped_file;

    fd = open("myfile.txt", O_RDWR); // Open the file
    if (fd == -1) {
        perror("open");
        exit(EXIT_FAILURE);
    }
    if (fstat(fd, &file_info) == -1) {     // Get the file size
        perror("fstat");
        exit(EXIT_FAILURE);
    }
    off_t file_size = file_info.st_size;
    mapped_file = mmap(NULL, file_size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (mapped_file == MAP_FAILED) {
        perror("mmap");
        exit(EXIT_FAILURE);
    }

    for (off_t i = 0; i < file_size; i++) {
        mapped_file[i] = toupper(mapped_file[i]);
    }

    if (munmap(mapped_file, file_size) == -1) {
        perror("munmap");
        exit(EXIT_FAILURE);
    }

    if (close(fd) == -1) {
        perror("close");
        exit(EXIT_FAILURE);
    }
    return 0;
}
```

main.c | myfile.txt

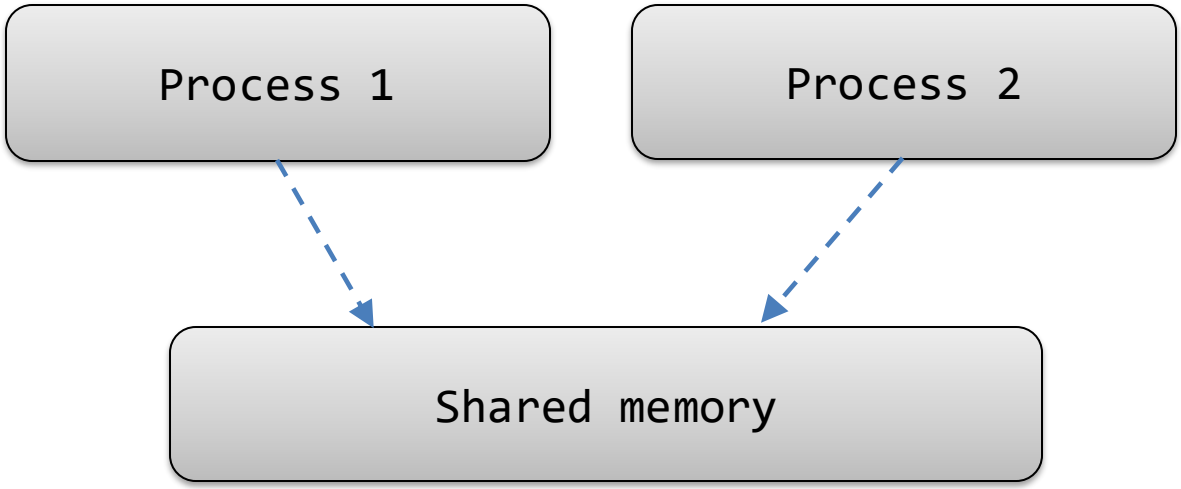
```
An operating system (OS) is system software that manages computer
hardware and software resources, and provides common services
for computer programs.
```

main.c | myfile.txt

```
AN OPERATING SYSTEM (OS) IS SYSTEM SOFTWARE THAT MANAGES COMPUTER
HARDWARE AND SOFTWARE RESOURCES, AND PROVIDES COMMON SERVICES
FOR COMPUTER PROGRAMS.
```

IPC - Linux
SHARED MEMORY

A memory shared between two or more processes is called shared memory.



IPC - Linux
SHARED MEMORY

A memory shared between two or more processes is called shared memory.

```
Shared memory segments
    System V shared memory allows processes to share a region a memory (a "segment").
    POSIX shared memory is an alternative API for achieving the same result; see shm_over-
    view(7).

    The System V shared memory API consists of the following system calls:

    shmget(2)
           Create a new segment or obtain the ID of an existing segment. This call returns
           an identifier that is used in the remaining APIs.

    shmat(2)
           Attach an existing shared memory object into the calling process's address
           space.

    shmdt(2)
           Detach a segment from the calling process's address space.

    shmctl(2)
           Perform various control operations on a segment, including deletion.
```

IPC - Linux

SHARED MEMORY

```
#include <sys/ipc.h>
#include <sys/shm.h>
```

ftok : convert a pathname and a project identifier to a System V IPC key

```
key_t ftok(const char *pathname, int proj_id);
```

shmget : allocates a System V shared memory segment

```
int shmget(key_t key, size_t size, int shmflg);
```

IPC - Linux
SHARED MEMORY

shmat : attaches the shared memory segment to the address space of the calling process.

```
void *shmat(int shmid, const void *shmaddr, int shmflg);
```

Shmdt : detaches the shared memory segment from the address space of the calling process.

```
int shmdt(const void *shmaddr);
```

Shmctl : performs the control operations on the shared memory segment

```
int shmctl(int shmid, int cmd, struct shmid_ds *buf);
```

IPC - Linux
SHARED MEMORY
POSIX

shm_open	create and open a new object, or open an existing object.
ftruncate	set the size of the shared memory object. (A newly created shared memory object has a length of zero.)
mmap	map the shared memory object into the virtual address space of the calling process.
shm_unlink	unmap the shared memory object from the virtual address space of the calling process.
close	close the file descriptor allocated by shm_open when it is no longer needed.
fstat	obtain a stat structure that describes the shared memory object.
fchown	to change the ownership of a shared memory object.
fchmod	to change the permissions of a shared memory object.

IPC - Linux

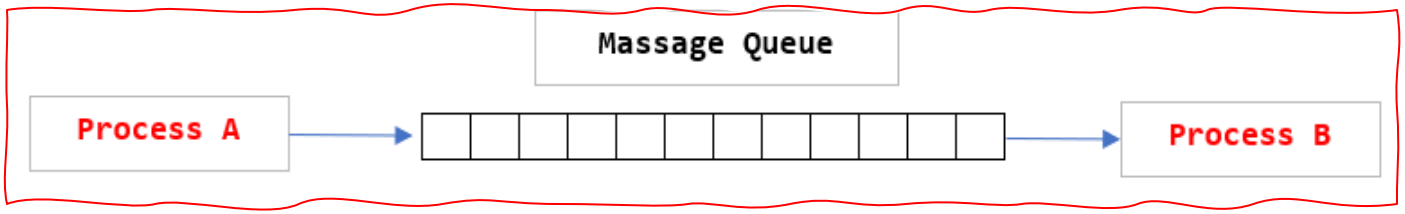
0.IPCS
1.PIPES and NAMED PIPES
2.SHARED MEMORY and MEMORY MAPPING
3.MESSAGE QUEUES
4.SEMAPHORES and NAMED SEMAPHORES
5.SIGNALS
6.SOCKET

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux
MESSAGE QUEUES

Message queues

System V message queues allow data to be exchanged in units called messages. Each messages can have an associated priority, POSIX message queues provide an alternative API for achieving the same result; see **mq_overview**(7).



The System V message queue API consists of the following system calls:

msgget(2)
Create a new message queue or obtain the ID of an existing message queue. This call returns an identifier that is used in the remaining APIs.

msgsnd(2)
Add a message to a queue.

msgrcv(2)
Remove a message from a queue.

msgctl(2)
Perform various control operations on a queue, including deletion.

IPC - Linux
MESSAGE QUEUES
POSIX

mq_open	creates a new POSIX message queue or opens an existing queue.
mq_close	close a message queue descriptor
mq_unlink	remove a message queue
mq_send	send a message to a message queue
mq_receive	receive a message from a message queue
mq_notify	register for notification when a message is available
mq_getattr,mq_setattr	get/set message queue attributes

IPC - Linux
MESSAGE QUEUES

Terminal - asp@asp-VirtualBox: ~/Desktop/OS/temp

File Edit View Terminal Tabs Help

```
~/Desktop/OS/temp$ ./ww
Message Queue is ready to send messages.
Enter lines of text, enter 'end' to quit:
MSG 1
MSG 2
MSG 3
MSG 4
MSG 5
```

Terminal - asp@asp-VirtualBox: ~/Desktop/OS/temp

File Edit View Terminal Tabs Help

```
~/Desktop/OS/temp$ ipcs -q

------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x42051a1a 1          asp        777        35           5
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux
MESSAGE QUEUES

Terminal - asp@asp-VirtualBox: ~/Desktop/OS/temp

File Edit View Terminal Tabs Help

```
~/Desktop/OS/temp$ ./rr
Message Queue is ready to receive messages.
received: "MSG 1"
received: "MSG 2"
received: "MSG 3"
received: "MSG 4"
received: "MSG 5"
```

Terminal - asp@asp-VirtualBox: ~/Desktop/OS/temp

File Edit View Terminal Tabs Help

```
~/Desktop/OS/temp$ ipcs -q

------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x42051a1a 1          asp        777        35           5

asp@asp-VirtualBox:~/Desktop/OS/temp$ ipcs -q

------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
0x42051a1a 1          asp        777        0            0
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux

0.IPCS
1.PIPES and NAMED PIPES
2.SHARED MEMORY and MEMORY MAPPING
3.MESSAGE QUEUES
4.SEMAPHORES and NAMED SEMAPHORES
5.SIGNALS
6.SOCKET

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux
NAMED/UNNAMED SEMAPHORES

Named semaphores:

- Identified by a name (e.g /my_semaphore), and can be used to synchronize multiple processes

Unnamed semaphores (memory based) :

- Used to synchronize threads within the same process

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux

NAMED/UNNAMED SEMAPHORES

```
down(s) ---> sem_wait(&s)
up(s)   ---> sem_post(&s)
```

```c
1  #include <pthread.h>
2  #include <semaphore.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  sem_t s;
7  int sem_value;
8
9  void critical_region(){
10   printf("Inside critical region.\n");
11 }
12
13 int main(){
14     sem_init(&s, 0, 1);
15
16     sem_getvalue(&s, &sem_value);
17     printf("Line %d, Semaphore value before sem_wait: %d \n", __LINE__, sem_value);
18     sem_wait(&s);
19     sem_getvalue(&s, &sem_value);
20     printf("Line %d, Semaphore value after  sem_wait: %d \n", __LINE__, sem_value);
21     critical_region();
22     sem_post(&s);
23     sem_getvalue(&s, &sem_value);
24     printf("Line %d, Semaphore value after sem_post : %d \n", __LINE__, sem_value);
25  }
```

```
Line 17, Semaphore value before sem_wait: 1
Line 20, Semaphore value after  sem_wait: 0
Inside critical region.
Line 24, Semaphore value after sem_post : 1
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux
NAMED/UNNAMED SEMAPHORES

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <semaphore.h>

int main()
{
    sem_t* semaphore = sem_open("/mySem", O_CREAT, 0644, 1);  // create named semaphore
    if (semaphore == SEM_FAILED)
    {
        perror("Error creating semaphore.\n");
        return 1;
    }
    sem_wait(semaphore);  // down(s)
    printf("Entered critical section.\n");
    sleep(100);
    sem_post(semaphore);  // up(s)
    sem_close(semaphore);  // close semaphore
    sem_unlink("/mySem");  // remove named semaphore
    return 0;
}

```

```
asp@asp-VB:/$ ls  /dev/shm/
sem.mySem
asp@asp-VB:/$ hexdump /dev/shm/sem.mySem
0000000 0000 0000 0000 0000 0080 0000 0000 0000
0000010 0000 0000 0000 0000 0000 0000 0000 0000
0000020
asp@asp-VB:/$ ls  /dev/shm/
asp@asp-VB:/$
```

location

removed

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC – Linux
SEMAPHORES
SYSTEM-V

```
Semaphore sets
    System  V  semaphores  allow processes to synchronize their actions System V semaphores
    are allocated in groups called sets; each semaphore in a set is a  counting  semaphore.
    The System V semaphore API consists of the following system calls:

    semget(2)
           Create a new set or obtain the ID of an existing  set.   This  call  returns  an
           identifier that is used in the remaining APIs.

    semop(2)
           Perform operations on the semaphores in a set.

    semctl(2)
           Perform various control operations on a set, including deletion.
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC – Linux
SEMAPHORES
POSIX

```
sem_wait, sem_timedwait, sem_trywait - lock a semaphore

    #include <semaphore.h>

    int sem_wait(sem_t *sem);
    int sem_trywait(sem_t *sem);
    int sem_timedwait(sem_t *sem, const struct timespec *abs_timeout);
```

sem_wait() decrements (locks) the semaphore pointed to by sem. If the semaphore's value is greater than zero, then the decrement proceeds, and the function returns, immediately. If the semaphore currently has the value zero, then the call blocks until either it becomes possible to perform the decrement (i.e., the semaphore value rises above zero), or a signal handler interrupts the call.

```
sem_post - unlock a semaphore

    #include <semaphore.h>

    int sem_post(sem_t *sem);
```

sem_post() increments (unlocks) the semaphore pointed to by sem. If the semaphore's value consequently becomes greater than zero, then another process or thread blocked in a **sem_wait**(3) call will be woken up and proceed to lock the semaphore.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC - Linux

0.IPCS
1.PIPES and NAMED PIPES
2.SHARED MEMORY and MEMORY MAPPING
3.MESSAGE QUEUES
4.SEMAPHORES and NAMED SEMAPHORES
5.SIGNALS
6.SOCKET

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC – Linux

SIGNALS

A signal is a software interrupt (or an event) generated by the system (e.g. divide by zero and so on) or generated by the user programmatically in response to an action or a condition.

kill -l

signal number(3)

signal (SIGQUIT)

```
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

IPC – Linux

SIGNALS

ctrl+Z **SIGSTOP(19)**
ctrl+c **SIGINT(2)**

```
~$ kill -l | grep SIGSTOP
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
~$ kill -l | grep SIGINT
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
```

```
~$ sleep 300000 &
[1] 491
~$ jobs
[1]+  Running                 sleep 300000 &
~$ kill -s SIGSTOP 491

[1]+  Stopped                 sleep 300000
~$ kill -s SIGCONT  491
~$ jobs
[1]+  Running                 sleep 300000 &
~$ kill -s SIGKILL   491
~$ jobs
[1]+  Killed                  sleep 300000
```

IPC – Linux
SIGNALS

A raised signal:

- Can be handled by user
- Can be ignored (**SIG_IGN**)
- Can be handled by default action (**SIG_DFL**)

```
SIGNAL(2)                        Linux Programmer's Manual                        SIGNAL(2)

NAME
       signal - ANSI C signal handling

SYNOPSIS
       #include <signal.h>

       typedef void (*sighandler_t)(int);

       sighandler_t signal(int signum, sighandler_t handler);

DESCRIPTION

       signal() sets the disposition of the signal signum to handler, which is either SIG_IGN,
       SIG_DFL, or the address of a programmer-defined function (a "signal handler").
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC – Linux

SIGNALS

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <signal.h>

void user_proc(int);
int main()
{
    pid_t pid;
    pid = getpid();
    signal(SIGINT, user_proc);
    while (1)
    {
        printf("pid = %d\n", pid);
        printf("First example!\n");
        sleep(1);
    }
    return 0;
}

void user_proc(int signum)
{

    printf("\nThe caught signal number is %d. \n", signum);
    exit(1);
}
```

```
First example!
pid = 1121
First example!
pid = 1121
First example!

The caught signal number is 2.
~$
~$
```

```
~$ kill -s SIGINT  1121
~$
~$
```

```
~$ ./a.out
pid = 1275
First example!
pid =
First example!
^C
The caught signal number is 2.
~$
```

ctrl+c

IPC – Linux

SIGNALS

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <signal.h>

void user_proc(int);
int main()
{
    pid_t pid;
    pid = getpid();
    signal(SIGINT, SIG_IGN);
    while (1)
    {
        printf("pid = %d\n", pid);
        printf("First example!\n");
        sleep(1);
    }
    return 0;
}

void user_proc(int signum)
{

    printf("\nThe caught signal number is %d. \n", signum);
    exit(1);
}
```

```
pid = 1680
First example!
Killed
~$
```

```
~$ kill -2   1680
~$ kill -9   1680
~$
```

IPC – Linux

SIGNALS

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <signal.h>

void user_proc(int);
int main()
{
    pid_t pid;
    pid = getpid();
    signal(SIGINT, SIG_DFL);
    while (1)
    {
        printf("pid = %d\n", pid);
        printf("First example!\n");
        sleep(1);
    }
    return 0;
}

void user_proc(int signum)
{

    printf("\nThe caught signal number is %d. \n", signum);
    exit(1);
}
```

```
pid = 1813
First example!
pid = 1813
First example!

~$
```

```
~$ kill -2   1813
~$
```

```
~$ ./a.out
pid = 1877
First example!
pid = 1877
Fi
^C
~$
```

ctrl+c

IPC - Linux

0.IPCS
1.PIPES and NAMED PIPES
2.SHARED MEMORY and MEMORY MAPPING
3.MESSAGE QUEUES
4.SEMAPHORES and NAMED SEMAPHORES
5.SIGNALS
6.SOCKET

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

IPC – Linux
SOCKET

```
SOCKET(2)                                                    Linux Programmer's Manual

NAME
       socket - create an endpoint for communication
       returns a file descriptor that refers to that endpoint.
SYNOPSIS
       #include <sys/types.h>          /* See NOTES */
       #include <sys/socket.h>

       int socket(int domain, int type, int protocol);
```

Parameter	Value	Description
Domain	AF_UNIX	Local communication
	AF_LOCAL	Synonym for AF_UNIX
	AF_INET	IPv4 Internet protocols
	AF_INET6	v6 Internet protocols
Type	SOCK_STREAM	Provides sequenced, reliable, two-way, connection-based byte streams.
	SOCK_DGRAM	Supports datagrams (connectionless, unreliable messages of a fixed maximum length)
	SOCK_RDM	Provides a reliable datagram layer that does not guarantee ordering.
	SOCK_RAW	Provides raw network protocol access.
Protocol		0

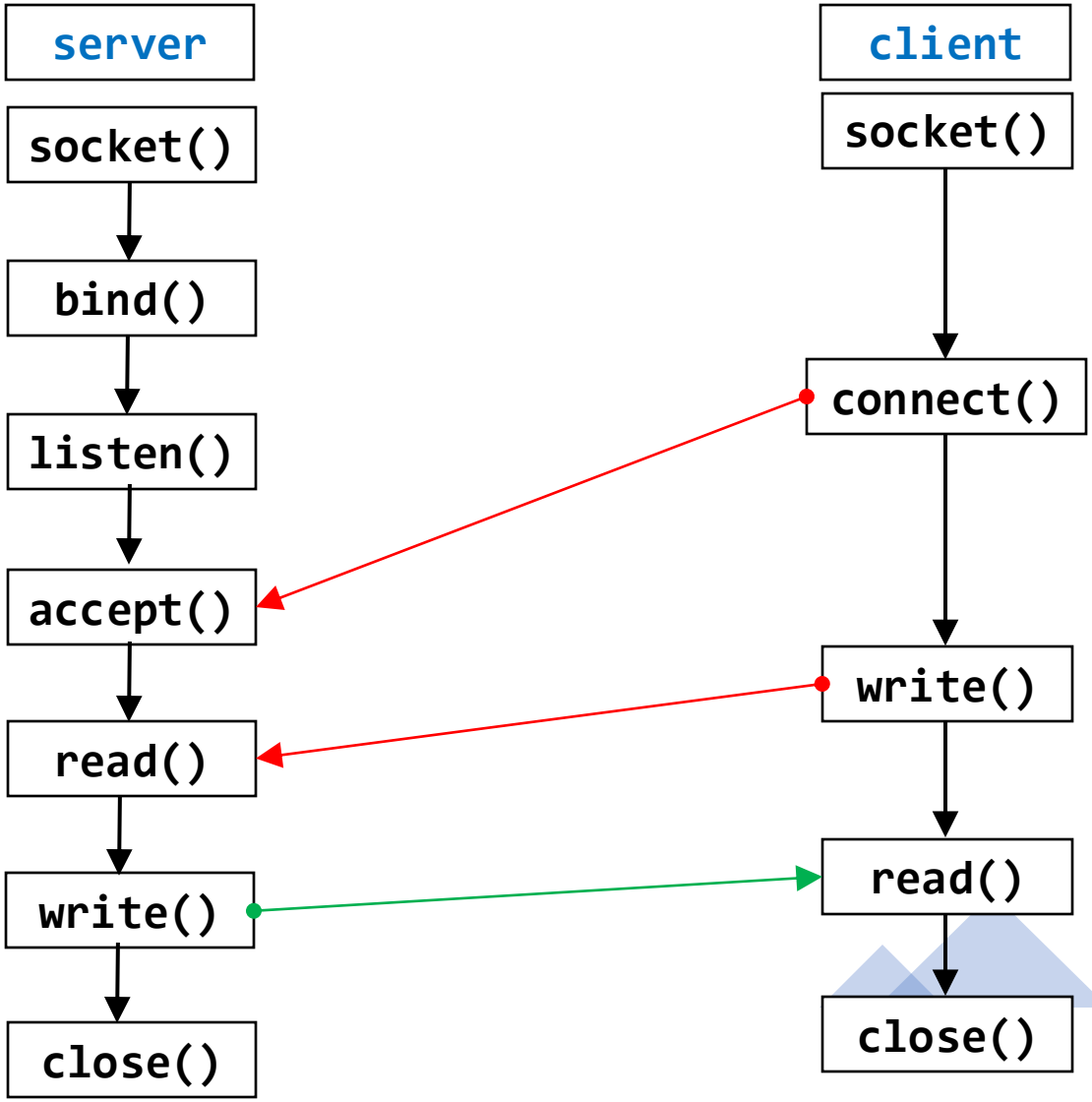
TCP

UDP

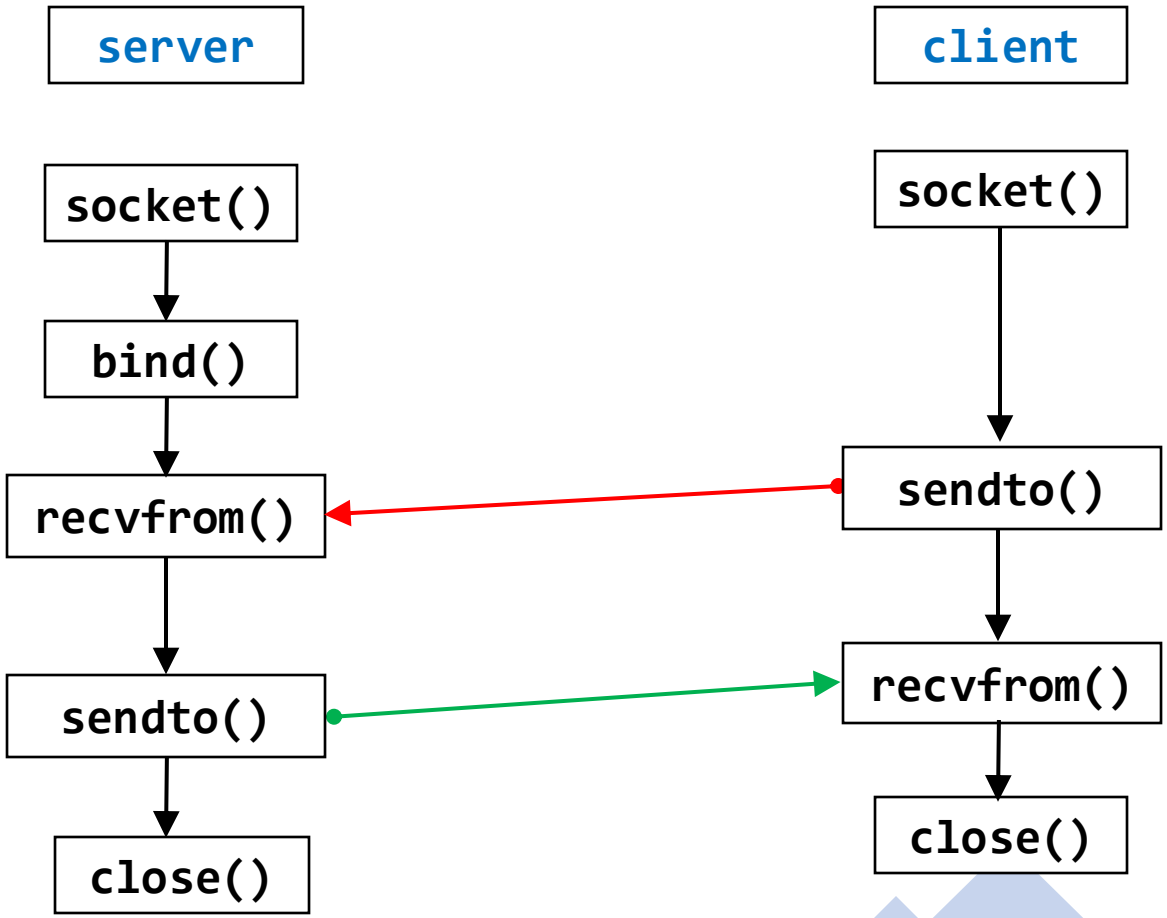
```
~$ cat  /etc/protocols
# Internet (IP) protocols
#
# Updated from http://www.iana.org/assignments/protocol-numbers and other
# sources.
# New protocols will be added on request if they have been officially
# assigned by IANA and are not historical.
# If you need a huge list of used numbers please install the nmap package.

ip      0       IP              # internet protocol, pseudo protocol number
hopopt  0       HOPOPT          # IPv6 Hop-by-Hop Option [RFC1883]
icmp    1       ICMP            # internet control message protocol
igmp    2       IGMP            # Internet Group Management
ggp     3       GGP             # gateway-gateway protocol
ipencap 4       IP-ENCAP        # IP encapsulated in IP (officially ``IP'')
```

IPC – Linux

SOCKET-TCP



IPC – Linux
SOCKET-UDP



The socketpair() system call facilitates the creation of a Unix socket that enables bidirectional communication, allowing data to be transmitted and received in both directions.

```
SOCKETPAIR(2)              Linux Programmer's Manual              SOCKETPAIR(2)

NAME
       socketpair - create a pair of connected sockets

SYNOPSIS
       #include <sys/types.h>          /* See NOTES */
       #include <sys/socket.h>

       int socketpair(int domain, int type, int protocol, int sv[2]);

DESCRIPTION
       The  socketpair()  call  creates an unnamed pair of connected sockets in
       the specified domain, of the specified type, and  using  the  optionally
       specified  protocol.   For  further  details  of  these  arguments,  see
       socket(2).

       The file descriptors used in referencing the new sockets are returned in
       sv[0] and sv[1].  The two sockets are indistinguishable.
```

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/socket.h>

int BUF_SIZE = 10;
int main( ) {
    int sock[2];
    char buf[BUF_SIZE];
    if (socketpair(PF_LOCAL, SOCK_DGRAM, 0, sock) < 0) {
        perror("socketpair failed!");
        return 1;
   }
   for (int i = 0; i < 10; i += 2) {
       sleep(1);
       sprintf(buf, "Message %d", i);
       write(sock[1], buf, sizeof(buf));
       read( sock[0], buf, BUF_SIZE);
       printf("-> %s \n", buf);
   }
   close(sock[0]);
   close(sock[1]);
   return 0;
   }
```

```
-> Message 0
-> Message 2
-> Message 4
-> Message 6
-> Message 8
```

Domain:

The domain argument specifies a communication domain; this selects the protocol family which will be used for communication. These families are defined in <sys/socket.h>. The formats currently understood by the Linux kernel include:

Name	Purpose	Man page
AF_UNIX	Local communication	**unix**(7)
AF_LOCAL	Synonym for **AF_UNIX**	
AF_INET	IPv4 Internet protocols	**ip**(7)
AF_AX25	Amateur radio AX.25 protocol	**ax25**(4)
AF_IPX	IPX - Novell protocols	
AF_APPLETALK	AppleTalk	**ddp**(7)
AF_X25	ITU-T X.25 / ISO-8208 protocol	**x25**(7)
AF_INET6	IPv6 Internet protocols	**ipv6**(7)
AF_DECnet	DECet protocol sockets	
AF_KEY	Key management protocol, originally developed for usage with IPsec	
AF_NETLINK	Kernel user interface device	**netlink**(7)
AF_PACKET	Low-level packet interface	**packet**(7)
AF_RDS	Reliable Datagram Sockets (RDS) protocol	**rds**(7) **rds-rdma**(7)
AF_PPPOX	Generic PPP transport layer, for setting up L2 tunnels (L2TP and PPPoE)	
AF_LLC	Logical link control (IEEE 802.2 LLC) protocol	
AF_IB	InfiniBand native addressing	
AF_MPLS	Multiprotocol Label Switching	
AF_CAN	Controller Area Network automotive bus protocol	

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.